ASSIGNMENT – 09

NAME : SURENDRA REDDY

HALL TICKET NO : 2303A51981

BATCH : 15

Question :

Expected Time to complete 1 Interacting with DeFi Protocols (Testnet) Objective To interact with a decentralized finance (DeFi) protocol on a blockchain testnet by writing and executing a smart contract that deposits and withdraws tokens, demonstrating basic DeFi operations without real funds.

Requirements

• Install Node.js and npm

• Install VS Code

• Install VS Code extensions: o Solidity o Hardhat

• Use a testnet (Sepolia / Goerli / Mumbai) •

Create a smart contract and a script to interact with a DeFi protocol (mock lending contract) • Use functions for deposit and withdrawal

• Test interactions using test tokens • Add inline comments explaining the logic

Tools & Technologies

• Solidity

• Hardhat

• JavaScript (Node.js)

• Ethereum Testnet (Sepolia)

• MetaMask (Testnet account) Practical Description In this practical, we simulate interaction with a DeFi protocol by creating a simple lending contract where:

• Users can deposit ETH

• Users can withdraw ETH

• Contract tracks balances internally This represents how DeFi protocols like Aave or Compound work on testnets.

CODE :

```
import tkinter as tk from tkinter

import messagebox class

DeFiLending:    def
```

```
__init__(self):       self.balances
= {}

  def deposit(self, user, amount):
if amount <= 0:
          return "Deposit must be greater than
0"        if user not in self.balances:
self.balances[user] = 0        self.balances[user]
+= amount        return f"{user} deposited
{amount} ETH"     def withdraw(self, user,
amount):        if user not in self.balances:
return "User does not exist"        if amount <=
0:
          return "Withdrawal must be greater than
0"        if self.balances[user] < amount:
return "Insufficient balance"
self.balances[user] -= amount        return f"{user}
withdrew {amount} ETH"     def get_balance(self,
user):
      return self.balances.get(user, 0)
class DeFiApp:
   def __init__(self, root):
      self.contract = DeFiLending()        self.root = root        self.root.title("DeFi
Lending Protocol Simulator")        self.root.geometry("400x350")
tk.Label(root, text="DeFi Simulator", font=("Arial", 16, "bold")).pack(pady=10)
tk.Label(root, text="User Name").pack()        self.user_entry = tk.Entry(root)
self.user_entry.pack()        tk.Label(root, text="Amount (ETH)").pack()
self.amount_entry = tk.Entry(root)

      self.amount_entry.pack()       tk.Button(root, text="Deposit",
command=self.deposit).pack(pady=5)       tk.Button(root, text="Withdraw",
command=self.withdraw).pack(pady=5)        tk.Button(root, text="Check Balance",
```

```
command=self.check_balance).pack(pady=5)        self.output = tk.Label(root, text="",
fg="blue")        self.output.pack(pady=10)     def deposit(self):
        user = self.user_entry.get()
        try:
            amount = float(self.amount_entry.get())
result = self.contract.deposit(user, amount)
self.output.config(text=result)        except:
            messagebox.showerror("Error", "Enter valid amount")
def withdraw(self):
        user = self.user_entry.get()
        try:
            amount = float(self.amount_entry.get())
result = self.contract.withdraw(user, amount)
self.output.config(text=result)        except:
            messagebox.showerror("Error", "Enter valid amount")


    def check_balance(self):
        user = self.user_entry.get()        balance =
self.contract.get_balance(user)
self.output.config(text=f"{user} Balance: {balance} ETH")
if __name__ == "__main__":
    root = tk.Tk()
    app = DeFiApp(root)
root.mainloop()
```

IMPLEMENTATION :

index.html 9+ gui.py

C: > Users > Rushi > Downloads > gui.py > DeFiApp > check_balance

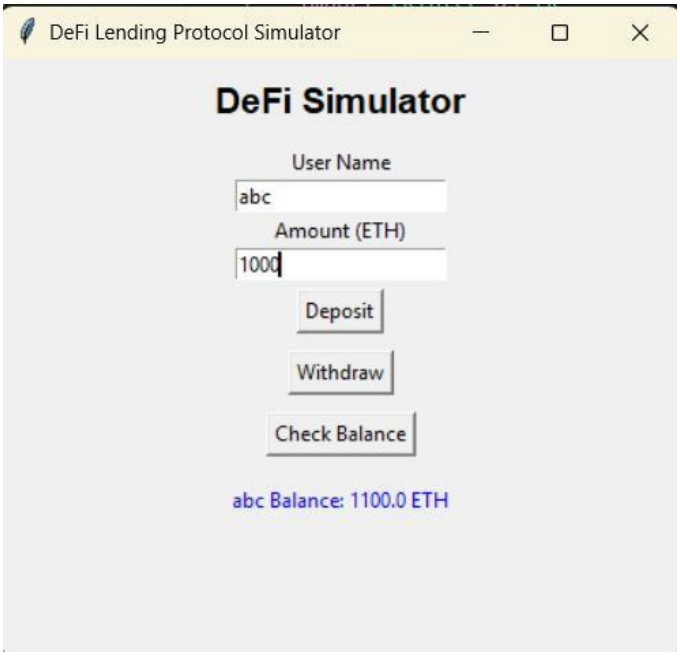
```python
import tkinter as tk
from tkinter import messagebox
class DeFiLending:
    def __init__(self):
        self.balances = {}
    def deposit(self, user, amount):
        if amount <= 0:
            return "Deposit must be greater than 0"
        if user not in self.balances:
            self.balances[user] = 0
        self.balances[user] += amount
        return f"{user} deposited {amount} ETH"
    def withdraw(self, user, amount):
        if user not in self.balances:
            return "User does not exist"
        if amount <= 0:
            return "Withdrawal must be greater than 0"
        if self.balances[user] < amount:
            return "Insufficient balance"
        self.balances[user] -= amount
        return f"{user} withdrew {amount} ETH"
    def get_balance(self, user):
        return self.balances.get(user, 0)
class DeFiApp:
    def __init__(self, root):
        self.contract = DeFiLending()
        self.root = root
        self.root.title("DeFi Lending Protocol Simulator")
        self.root.geometry("400x350")
        tk.Label(root, text="DeFi Simulator", font=("Arial", 16, "bold")).pack(pady=10)
        tk.Label(root, text="User Name").pack()
        self.user_entry = tk.Entry(root)
        self.user_entry.pack()
```

C: > Users > Rushi > Downloads > gui.py > DeFiApp > check_balance

```python
class DeFiApp:
    def __init__(self, root):
        tk.Button(root, text="Withdraw", command=self.withdraw).pack(pady=5)
        tk.Button(root, text="Check Balance", command=self.check_balance).pack(pady=5)
        self.output = tk.Label(root, text="", fg="blue")
        self.output.pack(pady=10)
    def deposit(self):
        user = self.user_entry.get()
        try:
            amount = float(self.amount_entry.get())
            result = self.contract.deposit(user, amount)
            self.output.config(text=result)
        except:
            messagebox.showerror("Error", "Enter valid amount")
    def withdraw(self):
        user = self.user_entry.get()
        try:
            amount = float(self.amount_entry.get())
            result = self.contract.withdraw(user, amount)
            self.output.config(text=result)
        except:
            messagebox.showerror("Error", "Enter valid amount")

    def check_balance(self):
        user = self.user_entry.get()
        balance = self.contract.get_balance(user)
        self.output.config(text=f"{user} Balance: {balance} ETH")
if __name__ == "__main__":
    root = tk.Tk()
    app = DeFiApp(root)
    root.mainloop()
```

OUTPUT :


DeFi Lending Protocol Simulator
DeFi Simulator
User Name
abc
Amount (ETH)
1000
Deposit
Withdraw
Check Balance
abc Balance: 1100.0 ETH